Tarea C — Recuperación léxica contra semántica latente sobre un corpus mínimo

Objetivo

Comparar un recuperador léxico (TF-IDF + similitud coseno) con su proyección semántica latente (LSA mediante `TruncatedSVD` con k=4) sobre un corpus de 10 documentos y 6 consultas con un documento relevante cada una (q1→d04, q2→d03, q3→d08, q4→d05, q5→d09, q6→d02). Se evalúa Hit@1, Hit@3 y MRR, se identifican las consultas falladas por cada método y se analiza el solapamiento léxico consulta–documento relevante, como base para discutir qué aportaría un recuperador denso en q5.

Método

Corpus, consultas y juicios cargados íntegramente en código. **TF-IDF**: `TfidfVectorizer` de scikit-learn con parámetros por defecto (minúsculas, `token_pattern \b\w\w+\b`, sin stopwords, idf suavizado, norma l2); el vocabulario resultante tiene 105 términos. Se calcula similitud coseno consulta–documento y se ordenan los 10 documentos por consulta. **LSA**: `TruncatedSVD(n_components=4, random_state=0)` sobre la misma matriz TF-IDF, repitiendo ranking y evaluación en el espacio latente. **Métricas** sobre las 6 consultas: Hit@1, Hit@3 y MRR (media de rangos recíprocos). Criterio de fallo: relevante fuera del top 3. Para el análisis de fallos se intersectan los tokens de la consulta y de su relevante con el mismo tokenizador del vectorizador, separando términos de contenido de funcionales.

Resultados

Comparativa global (Parte 2):

Método	Hit@1	Hit@3	MRR
TF-IDF	0.8333	0.8333	0.8611
LSA (k=4)	0.6667	0.8333	0.75

La proyección a 4 dimensiones explica 0.3953 de la varianza (componentes: 0.0342, 0.1266, 0.1160, 0.1186). LSA pierde 0.1667 en Hit@1 y 0.1111 en MRR; Hit@3 no cambia. En TF-IDF, las cinco consultas resueltas colocan el relevante en el puesto 1 con similitudes entre 0.4569 (q4) y 0.6891 (q1).

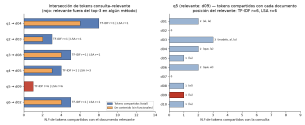
Recuperación sobre 6 consultas: TF-IDF vs LSA (k=4)

	Hit@1	Hit@3	MRR
TF-IDF	0.8333	0.8333	0.8611
LSA (TruncatedSVD, k=4)	0.6667	0.8333	0.7500

Análisis por consulta (Parte 3) — rank del relevante y términos de contenido compartidos consulta–relevante:

Consulta	Relevante	Contenido compartido	Rank TF-IDF	Rank LSA
q1	d04	6 (función, ranking, léxica, pondera, frecuencia, términos)	1	1
q2	d03	2 (fragmentos, prompt)	1	1
q3	d08	4 (entrena, matrices, bajo, rango)	1	1
q4	d05	3 (compara, orientación, vectores)	1	3
q5	d09	0 (solo comparten «la»)	6	6
q6	d02	4 (autoatención, capas, feed, forward)	1	1

Ambos métodos fallan únicamente q5 (rank 6, rango recíproco 0.1667). LSA además degrada q4 del puesto 1 al 3: sigue en el top 3, pero explica la caída de Hit@1 y MRR. En q5 con TF-IDF, d09 alcanza similitud 0.0790, por detrás de d03 (0.3107), d06 (0.2009), d04 (0.1896), d01 (0.1623) y d10 (0.0849); d03 supera a d09 porque comparte «modelo» (más «al», «la»), mientras que con d09 la consulta solo comparte el artículo «la».



Discusión

Por qué falla q5 en ambos métodos. La consulta pregunta por determinismo («determinista», «elegir», «palabra», «modelo») y d09 habla de «temperatura», «logits», «softmax» y «probabilidad»: el solapamiento de contenido es nulo (solo comparten «la»). TF-IDF no puede emparejarlos y ordena por coincidencias incidentales («modelo» en d03, artículos en d01/d04/d10). LSA tampoco lo corrige: su espacio latente se aprende por co-ocurrencia dentro del corpus, y en 10 documentos «determinista» nunca co-ocurre con «temperatura» o «softmax», de modo que la asociación semántica simplemente no existe en los datos. Además, comprimir a k=4 retiene solo 0.3953 de la varianza, lo que produce el retroceso de d05 en q4 (1→3): la proyección diluye matching léxico que sí funcionaba.

Qué aportaría un recuperador denso. Un codificador entrenado aprende, en corpus masivos, que «más determinista» es paráfrasis de «temperatura baja concentra la probabilidad» y que «softmax» es el mecanismo implicado, aunque consulta y documento no compartan ni un término de contenido. La similitud coseno en ese espacio captaría sinonimia y paráfrasis, colocando d09 en el top, algo fuera del alcance de TF-IDF y de LSA entrenada por co-ocurrencia en un corpus mínimo.

Costos frente a TF-IDF. (1) Datos y entrenamiento: exige un modelo preentrenado o ajustado, frente a un TF-IDF sin entrenamiento; (2) latencia y cómputo: codificar consultas y documentos con una red neuronal cuesta mucho más que un producto punto sobre vectores dispersos de 105 términos; (3) memoria: hay que almacenar el modelo y embeddings densos de alta dimensión por documento; (4) interpretabilidad: el fallo de q5 en TF-IDF es auditable (solo comparten «la»), mientras que en un espacio denso no pueden señalarse términos compartidos. A cambio, el recuperador denso generaliza ante brechas de vocabulario exactamente como la de q5, el caso donde ambos métodos de esta tarea fallan.